Practice Exam #3

SECTION I

Time — 90 minutes
Number of questions — 42
Percent of total grade — 55

1. Consider the following code segment.

```
if (!somethingIsFalse())
{
  return false;
}
else
{
  return true;
}
```

Which of the following replacements for this code will produce the same result?

(A) `return true;`
(B) `return false;`
(C) `return somethingIsFalse();`
(D) `return !somethingIsFalse();`

2. Consider the following code segment.

```
String abc = "AAABBBCCC";
String abc1 = abc.substring(0, abc.length() - 1);

abc1 = abc1.substring(1, abc1.length());
System.out.println(abc.indexOf(abc1));
```

What is printed when the code segment is executed?

(A) `-1`
(B) `0`
(C) `1`
(D) `StringIndexOutOfBoundsException`

3. Consider the following statements.

```
double x = 2.5, y = 1.99;
System.out.println((int)(x/y) + (int)(x*y));
```

What is printed when the statements are executed?

(A) 3
(B) 4
(C) 4.0
(D) 5

4. Assume that x and y are `boolean` variables that have been properly declared and initialized. When do the logical expressions `!x && (x || y)` and `x || !y` have the same value?

(A) Never
(B) Only when x is `true`
(C) Only when x is `false`
(D) Only when both x and y are `false`

5. Consider the following code segment.

```
Integer year = 2026;
System.out.println(year + Integer.parseInt("1"));
```

What is the result when the code segment is compiled/executed?

(A) Syntax error
(B) 2027
(C) 20261
(D) "2027"

6. Consider the following code segment.

```
int a = 3;
int b = a % 3;
a += b;
if (a > 3 && a/b < 2)
{
  System.out.println(a/b);
}
```

What is printed when the code segment is executed?

(A) The value of `Integer.MAX_VALUE`
(B) `java.lang.ArithmeticException: / by zero`
(C) `3`
(D) Nothing

7. For which values of the `boolean` variables `x` and `y` does the expression

```
(!(x && y) == (!x || !y)) && (!(x || y) == (!x && !y))
```

evaluate to `true`?

(A) For any values
(B) Only when both `x` and `y` are `true`
(C) Only when both `x` and `y` are `false`
(D) Never

8. Consider the following statement.

```
System.out.println("\\\"hello\"".substring(1));
```

What is printed when the statement is executed?

(A) `"hello"`
(B) `hello`
(C) `\\hello`
(D) `\\"hello"`

9. Consider the following code segment.

```
int n = 1, d = 1, s = 1;

while (n <= 5)
{
  System.out.print(s + " ");
  n++;
  d += 2;
  s += d;
}
System.out.println();
```

What is printed when the code segment is executed?

(A) 1 3 5 7
(B) 1 3 6 10 15
(C) 1 4 9 16 25
(D) 1 4 7 10 13

10. Consider the following method.

```
public void reduce(int[] arr, int len)
{
  for (int k = 0; k < len; k++)
  {
    arr[k]--;
  }
  len--;
}
```

What is printed when the following code segment is executed?

```
int[] counts = {3, 2, 1, 0};
int len = 3;
reduce(counts, len);

for (int c : counts)
{
  System.out.print(c + " ");
}

System.out.println(len);
```

(A) 2 1 0 0 3
(B) 2 1 0 0 2
(C) 2 1 1 0 3
(D) 2 1 0 -1 2

11. Suppose a class `Particle` has the following fields defined.

```
public class Particle
{
  public static final int START_POS = 100;
  private double velocity;

  private boolean canMove()
  { /* implementation not shown */ }

  /* constructors, other fields and methods not shown */
}
```

Which of the following is true?

(A) Java syntax rules wouldn't allow us to use the name `startPos` instead of `START_POS`.

(B) Both `velocity` and `START_POS` can be changed by one of `Particle`'s methods.

(C) A statement

```
double pos = START_POS + velocity;
```

in `Particle`'s `canMove` method would result in a syntax error.

(D) None of the above

12. Consider the following method.

```
public int randomPoints(int n)
{
  return (int)(n * Math.random()) + 1;
}
```

Which of the following outputs is NOT possible when the statement

```
System.out.println(randomPoints(3) + randomPoints(3));
```

is executed?

(A) 2
(B) 4
(C) 6
(D) All of the above are possible.

13. Consider the following method with two missing statements.

```
/**
 *  Returns the sum of all positive odd values
 *  among the first n elements of arr.
 *  Precondition: 1 <= n <= arr.length
 */
public static int addPositiveOddValues(int[] arr, int n)
{
  int sum = 0;

  < statement 1 >
  {
    < statement 2 >
      sum += arr[i];
  }
  return sum;
}
```

Which of the following are appropriate replacements for < *statement 1* > and < *statement 2* > so that the method works as specified?

	< *statement 1* >	< *statement 2* >
(A)	`for (int i = 1; i < n; i += 2)`	`if (arr[i] > 0)`
(B)	`for (int i = 0; i < n; i++)`	`if (arr[i] > 0 && arr[i] % 2 != 0)`
(C)	`for (int i = 1; i <= n; i += 2)`	`if (arr[i] > 0)`
(D)	`for (int i = 0; i <= n; i++)`	`if (arr[i] % 2 != 0)`

14. Consider the following method.

```
private double compute(int x, int y)
{
  double r = 0;

  if (!(y == 0 || x/y <= 2))
  {
    r = 1 / ((x - 2*y) * (2*x - y));
  }

  return r;
}
```

For which of the following values of `x` and `y` will `compute(x, y)` throw an exception?

(A) `x = 0, y = 0`
(B) `x = 1, y = 0`
(C) `x = 3, y = 5`
(D) None of the above

15. Consider the following code segment.

```
ArrayList<Integer> numbers = new ArrayList<Integer>();
numbers.add(0);
numbers.add(-1);
numbers.add(1);
numbers.add(-2);
numbers.add(-3);

int i = 0;
for (Integer x : numbers)
{
  if (x < 0)
  {
    numbers.remove(i);
  }
  i++;
}
System.out.println(numbers);
```

What is the outcome when this code is compiled and executed?

(A) `ConcurrentModificationException`
(B) `IndexOutOfBoundsException`
(C) The code runs with no errors, `[0, 1]` is printed
(D) The code runs with no errors, `[0, 1, -3]` is printed

16. What is printed when the following statement is executed?

```
int x = 1024;
System.out.println((1 - 2*x) / x);
```

(A) `-2`
(B) `-1`
(C) `0`
(D) `1`

17. Consider the following classes.

```
public class APTestResult
{
  private String subject;
  private int score;

  public int getScore()
  {
    return score;
  }

  /* constructors and other methods not shown */
}

public class APScholar
{
  private String name;
  private int id;
  private ArrayList<APTestResult> exams;

  public int getExamResult(k)
  {
    return exams.get(k);
  }

  /* constructors and other methods not shown */
}
```

Given

```
APScholar[] list = new APScholar[100];
```

in a client class, which of the following expressions correctly represents the third AP score of the first `APScholar` in `list`?

(A) `list[0].exams[2].getScore()`
(B) `list[2].getExamResult().getScore()`
(C) `list[0].getExamResult().getScore(2)`
(D) None of the above

18. What is the result when the following code segment is compiled/executed?

```
String a = "A";
String b = a;
String c = a + 1;     // Line 1
String d = b + "1";

System.out.println((c == d) + " " + c.equals(d));
```

(A) `false false` is printed
(B) `false true` is printed
(C) `true true` is printed
(D) Syntax error on Line 1

19. Consider the following code segment.

```
ArrayList<Integer> numbers = new ArrayList<Integer>();
numbers.add(0);
numbers.add(1);

for (int k = 1; k <= 3; k++)
{
  int n = numbers.size();
  for (int i = 0; i < n; i++)
  {
    numbers.add(numbers.get(i) + 1);
  }
}

int n = numbers.size();
System.out.println(n + " " + numbers.get(n-1));
```

What is printed when the code segment is executed?

(A) `8 3`
(B) `8 4`
(C) `16 3`
(D) `16 4`

20. Consider the following method `fun2`.

```
public int fun2(int x, int y)
{
  y -= x;
  return y;
}
```

What are the values of the variables `a` and `b` after the following code segment is executed?

```
int a = 3, b = 7;
b = fun2(a, b);
a = fun2(b, a);
```

(A) `-1` and `4`
(B) `-4` and `7`
(C) `-4` and `4`
(D) `3` and `7`

21. Suppose it takes about 18 milliseconds to sort an array of 80,000 random numbers using Mergesort. Suppose for an array of 160,000 numbers, Mergesort runs for 40 milliseconds. For approximately how much time will Mergesort run on an array of 320,000 numbers? Choose the closest estimate.

(A) 88 milliseconds
(B) 96 milliseconds
(C) 126 milliseconds
(D) 192 milliseconds

22. Consider the following code segment.

```
String abc = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
< missing statement >
System.out.println(abc.substring(k, k+1));
```

It is intended to print a randomly chosen letter of the alphabet. Any of the 26 letters can be chosen with equal probability. Which of the following can replace *< missing statement >* for the code to work that way?

(A) `int k = Math.random(0, 25);`
(B) `int k = Math.random(25) + 1;`
(C) `int k = (int)(26*Math.random());`
(D) `int k = (int)(25*Math.random()) + 1;`

23. Consider the following method.

```
public static void lol(int n)
{
  if (n == 0)
  {
    return;
  }

  lol(n-1);

  for (int k = 1; k <= n; k++)
  {
    System.out.print("LOL");
  }
}
```

When `lol(4)` is called, how many times will `lol` actually be called, including the initial call?

(A) 2
(B) 4
(C) 5
(D) 10

24. Suppose a `Scanner` object `scan` is associated with a text file open for reading. The following `while` loop is intended to print all the lines from the file.

```
while (scan.hasNext())
{
  System.out.println(scan.nextLine());
}
```

What happens when the first line in the file is a blank line?

(A) The program is aborted with an `InputMismatchException`.
(B) The `while` loop is exited after the first iteration and nothing is printed.
(C) The `while` loop runs through every line in the file, but the first line is not printed.
(D) A blank line is printed, and the loop continues normally.

25. What is the most likely value of the expression

```
(int)(Math.random() + Math.random() + 0.5)
```

(A) 0
(B) 1
(C) 2
(D) 1 and 2 are equally likely; 0 is less likely

26. Consider the following method.

```
public void process(int[] a)
{
  for (int i = 1; i < a.length; i++)
  {
    int current = a[i];
    int j = 0;

    while (a[j] < current)
    {
      j++;
    }

    for (int k = i; k > j; k--)
    {
      a[k] = a[k-1];
    }

    a[j] = current;
  }
}
```

The algorithm implemented in the method `process` is best described as:

(A) Mergesort
(B) Insertion Sort
(C) Selection Sort
(D) A faulty implementation of a sorting algorithm

27. Consider the following code segment.

```
int[] arr = {4, 3, 2, 1, 0};

for (int k = 1; k < arr.length; k++)
{
  arr[k-1] += arr[k];
}
```

Which of the following represents the contents of `arr` after the code segment has been executed?

(A) 4, 7, 5, 3, 1
(B) 7, 5, 3, 1, 0
(C) 7, 3, 2, 1, 0
(D) 10, 6, 3, 1, 0

28. What is printed when the following code segment is compiled and executed?

```
String s = "ABC";
for (int i = 0; i < s.length(); i++)
{
  s += s.substring(i);
}
System.out.println(s);
```

(A) ABCABC
(B) ABCAABABC
(C) ABCABCBCC
(D) Nothing due to memory overflow error

29. What is printed when the following code segment is executed?

```
ArrayList<Integer> list1 = new ArrayList<Integer>();
ArrayList<Integer> list2 = list1;
list1.add(1);
list2.add(0, 2);
System.out.println(list1 + " " + list2);
```

(A) [1] [2]
(B) [1, 2] [2]
(C) [1] [0, 2]
(D) [2, 1] [2, 1]

30. Ashanti has decided to write and test her own version of the `indexOf` method, which takes two strings as parameters:

```
/**
 *  Returns the first index k such that word.substring(k)
 *  starts with str; if no such k found, returns -1.
 *  Preconditions: word != null, str != null
 */
public int indexOf(String word, String str)
{
  int n = str.length();
  for (int k = 0; k <= word.length() - n; k++)
  {
    if (word.substring(k, k + n).equals(str))
    {
      return k;
    }
  }
  return -1;
}
```

What will happen if Morgan calls Ashanti's `indexOf` method as follows?

```
int i = indexOf("Hello", null);
```

(A) `i` will be set to 0
(B) `i` will be set to -1
(C) `NullPointerException`
(D) `StringIndexOutOfBoundsException`

31. The statement

```
System.out.println(Integer.MAX_VALUE);
```

prints `2147483647`. What does the following statement print?

```
System.out.println(Integer.MAX_VALUE / 10 + 1);
```

(A) `214748365`
(B) `214748365.7`
(C) `214748365.0`
(D) Nothing: it causes a syntax error

32. Consider the following method.

```
/**
 *  Returns the index of target in arr or -1 if not found.
 *  Preconditions: arr is sorted in ascending order;
 *                 arr contains no duplicates
 */
public static int find(String[] arr, String target)
{
  < code not shown >
}
```

Suppose an optimal search algorithm is used in the above `find` method based on String's `compareTo` calls. Suppose also that `names.length` is 5 and `names` satisfies `find`'s precondition. What is the worst-case number of calls to `String`'s `compareTo` method when `find(names, "Desiree")` is called?

(A) 2
(B) 3
(C) 4
(D) 5

33. What is printed when the following code segment is executed?

```
String[] xy = {"X", "Y"};
String[] yx = xy;
yx[0] = xy[1];
yx[1] = xy[0];

System.out.println(xy[0] + xy[1] + yx[0] + yx[1]);
```

(A) XXXX
(B) YYYY
(C) XYYX
(D) XYXY

34. Consider the following code segment.

```
ArrayList<String> list = new ArrayList<String>();
list.add("One");
list.add("Two");
String[] msg = new String[2];
list.add(msg[0]);
< another statement >
```

Which of the following choices for < *another statement* > will cause a `NullPointerException` when the code is compiled and executed?

(A) `msg[0] = "Three";`
(B) `msg[0] = list.get(list.size());`
(C) `msg[1] = msg[0].substring(0, 2);`
(D) `list.add(2, msg[0]);`

35. Given two arrays of `double` values sorted in ascending order, one with 100 elements and the other with 10 elements, how many comparisons will it take in an optimal algorithm to merge these arrays into one sorted array, in the best case and in the worst case?

	Best case	Worst case
(A)	10	109
(B)	50	110
(C)	100	110
(D)	109	999

36. Consider the following method.

```
public void mystery(int a, int b)
{
  System.out.print(a + " ");
  if (a <= b)
  {
    mystery(a + 5, b - 1);
  }
}
```

What is printed when `mystery(0, 16)` is called?

(A) `0`
(B) `0 5 10`
(C) `0 5 10 15`
(D) `0 5 10 15 20`

37. In a regular pentagon, the ratio of the length of a diagonal to the length of a side is equal to the Golden Ratio (defined as $\frac{1+\sqrt{5}}{2} \approx 1.618$). Consider the following class `Pentagon`, which represents a regular pentagon.

```
public class Pentagon
{
  public static final double GOLDEN_RATIO =
                                    (1 + Math.sqrt(5.0)) / 2;
  private double side;

  public Pentagon (double x)
  {
    side = x;
  }

  public double getDiagonalLength()
  {
    return side * GOLDEN_RATIO;
  }
}
```

Which of the following code segments will compile with no errors but fail to print the correct length of a diagonal in a regular pentagon with side 3.0?

(A)
```
Pentagon p = new Pentagon(3);
System.out.println(p.getDiagonalLength());
```
(B)
```
System.out.println(3.0 * Pentagon.GOLDEN_RATIO);
```
(C)
```
System.out.println(3*(1 + Math.sqrt(5.0))/2);
```

(D) All of the above will work fine.

38. At a county fair, prizes are awarded to the five heaviest cows. More than 2000 cows are entered, and their records are stored in an array. Which of the following algorithms provides the most efficient way of finding the records of the five heaviest cows?

(A) Selection Sort
(B) Insertion Sort
(C) Insertion Sort terminated after the first five iterations
(D) Selection Sort terminated after the first five iterations

39. Consider the following code segment.

```
int n = input.nextInt();    // read an int value
n = Math.abs(n);

while (n >= 2)
{
  n = n/2 - 1;
}
System.out.println(n);
```

Which of the following represents the list of all possible outputs?

(A) 0
(B) -1, 0
(C) 0, 1
(D) -1, 0, 1

40. Consider the following class.

```
public class Counter
{
  private int count = 0;

  public Counter() { count = 0; }
  public Counter(int x) { count = x; }              // Line 1
  public int getCount() { return count; }           // Line 2
  public void setCount(int c) { int count = c; }    // Line 3
  public void increment() { count++; }              // Line 4
  public String toString() { return "" + count; } // Line 5
}
```

The test code

```
Counter c = new Counter();
c.setCount(3);
c.increment();
System.out.println(c.getCount());
```

is supposed to print 4, but the class has an error. What is actually printed, and which line in the class definition should be changed to get the correct output, 4?

(A) 0, Line 1
(B) 1, Line 3
(C) 0, Line 4
(D) 3, Line 4

41. Consider the following method.

```
public static boolean isGood(int[][] m)
{
  for (int r = 0; r < m.length; r++)
  {
    if (r > 0 && m[r][0] < m[r-1][0])
    {
      return false;
    }

    for (int c = 1; c < m[0].length; c++)
    {
      if (m[r][c] < m[r][c-1])
      {
        return false;
      }
    }
  }
  return true;
}
```

For which of the following 2D arrays `m` does `isGood(m)` return `false`?

(A)
```
int[][] m = {{1, 3, 5, 7},
             {2, 3, 4, 8},
             {3, 4, 5, 6}};
```
(B)
```
int[][] m = {{0, 1, 5, 6},
             {1, 2, 3, 4},
             {3, 2, 1, 0}};
```
(C)
```
int[][] m = {{5,  10, 15, 20},
             {10, 15, 20, 25},
             {11, 12, 13, 14}};
```
(D)
```
int[][] m = {{5, 16, 17, 18},
             {7, 12, 20, 25},
             {8, 10, 13, 20}};
```

42. Spreadsheet software has data organized in a table of rows and columns, and it has commands that work with that table.

My spreadsheet contains information about 500 restaurants, one row per restaurant. In each row, the first column is the restaurant's name; the second column is its rating (the average number of stars, from 1 to 5); the third column is the number of reviews; the fourth column is the distance, in miles, from my home. Most of the restaurants have many reviews, but some, which opened recently, may have only a few. There are no more than 25 restaurants within 10 miles of my home.

Spreadsheet's *sort* command lets me choose any two columns (let's call them *A* and *B*) and first sort the spreadsheet by column *A*, in ascending or descending order, then sort by column *B* each group of rows that have the same value in column *A*, in ascending or descending order. Which setting for the *sort* command should I choose to help me find a restaurant within 10 miles of my home, rated at least four stars, that has at least 12 reviews?

(A) First by rating, descending; then by distance, ascending
(B) First by rating, descending; then by number of reviews, descending
(C) First by distance, ascending; then by rating, descending
(D) First by number of reviews, descending; then by distance, ascending

Practice Exam #3

SECTION II

Time — 90 minutes
Number of questions — 4
Percent of total grade — 45

1. Glogal Enterprises is running a survey to determine their customers' ages. It promises a prize to a randomly chosen survey participant. The survey results are kept in a text file, but you won't have to work with files directly in this question. The Java class `Sweepstakes` helps determine the sweepstakes winner. You will write two methods of the `Sweepstakes` class.

   A partial definition of the `Sweepstakes` class is shown below.

```
public class Sweepstakes
{
  /** The candidate to win sweepstakes */
  private String winner = "";

  /** The sequential number of the candidate */
  private int recordNumber = 0;

  /**
   *  Initializes a Scanner object associated
   *  with a text file open for reading.
   */
  public Sweepstakes(String pathname)
  { /* implementation not shown */ }

  /**
   *  Returns the current year.
   */
  public int currentYear()
  { /* implementation not shown */ }

  /**
   *  Returns the next record from the file, or
   *  null if no more records are found.
   */
  public String nextRecord()
  { /* implementation not shown */ }
```

Continued

```
    /**
     *  Returns a record randomly selected with equal
     *  probability among all the records in the sweepstakes
     *  file, as described in Part (a).
     */
    public String chooseWinner()
    { /* to be implemented in Part (a) */ }

    /**
     *  Analyzes the winner's record, removes the middle
     *  initial if present, calculates the winner's age,
     *  and returns a message in the format described
     *  in Part (b).
     */
    public String reportWinner()
    { /* to be implemented in Part (b) */ }

    /* other instance variables and methods not shown */
}
```

(a) Write the method `chooseWinner` that sets the instance variable `winner` to a randomly chosen record from the sweepstakes file. At first glance, it seems impossible to choose a random winner among all the records with equal probability without creating a list of all candidates first. However, there is a simple algorithm to accomplish this task by keeping track of the record number of each candidate. It works as follows.

Set the instance variable `winner` to an empty string and `recordNumber` to 0. When the next record is received, you increment `recordNumber` and update `winner`, but not always — only with the probability `1/recordNumber`. So, for the first record, `winner` is updated with the probability 1/1 (always); for the second record, `winner` is updated with the probability 1/2; for the third record, `winner` is updated with the probability 1/3; and so on. It is not very hard to prove mathematically that each record will be selected with the probability $1/n$, where n is the total number of records.

Complete the `chooseWinner` method below.

```
/**
 *  Returns a record randomly selected with equal
 *  probability among all the records in the sweepstakes
 *  file.
 */
public String chooseWinner()
```

(b) Each record in the sweepstakes file has the format first name, optional middle initial, last name, and date of birth as mm/dd/yyyy. For example,

```
Angel Reese 05/06/2002
```

or

```
Caitlin E. Clark 01/22/2002
```

The method `reportWinner` parses this string, removes the middle initial (if present), calculates the winner's age by subtracting their year of birth from the current year (which may cause it to be off by one, if the winner's birthday hasn't happened yet, but that's okay for our purposes), and removes the date of birth. For example, a message returned by `reportWinner` could look like this:

```
And the winner is... Caitlin Clark, age 24
```

Complete the `reportWinner` method below. Make sure the returned message has the exact format shown above.

```
/**
 *  Analyzes the winner's record, removes the middle
 *  initial if present, calculates the winner's age,
 *  and returns a message in the format shown in the
 *  examples.
 */
public String reportWinner()
```

Class information for this question

```
public class Sweepstakes

private String winner = ""
private int recordNumber

public Sweepstakes(String pathname)
public int currentYear()
public String nextRecord()
public String chooseWinner()
public String reportWinner()
```

For additional practice, complete the `Sweepstakes` class. Add an instance variable `Scanner scan` to the class and the following code to the constructor to create a `Scanner` object associated with the `Sweepstakes.txt` file.

```
File file = new File(pathname);

try
{
  scan = new Scanner(file);
}
catch (FileNotFoundException e)
{
  System.out.println("Can't find " + pathname);
  System.exit(1);
}
```

Test your class with the small `Sweepstakes.txt` file provided for your convenience. Then repeat the test 10,000 times with the same file, creating a new `Sweepstakes` object each time, and count how many times each record is chosen as the winner to make sure each record has roughly the same chance of being selected.

2. The `CarMileage` class, which you will write, tracks a car's fuel usage and average gas mileage. The `CarMileage` class defines a constant (`final double` instance variable) that represents the size of the car's gas tank (in gallons). `CarMileage`'s constructor initializes that constant to a value of the type `double`, passed to the constructor as a parameter.

   The `CarMileage` class contains a `milesPerGallon` method, which takes two parameters.

   - The first parameter is an `int` that represents the number of miles traveled. Assume that this value will be greater than 0.
   - The second parameter is a `double` that represents the fraction of the tank used on the trip. Assume that this value will be greater than 0.0 and will not exceed 1.0.

   `milesPerGallon` returns the average gas mileage (in miles per gallon) rounded to the nearest integer. This average is computed over all the trips entered in calls to this method.

   The following table contains a sample code execution sequence and the corresponding `results`. The code execution sequence appears in a class other than `CarMileage`.

Statement	Value Returned (blank if no value)	Explanation
`CarMileage car = new CarMileage(12.0);`		`CarMileage car` object is constructed, with a tank of 12.0 (gallons)
`car.milesPerGallon(65, 1/4.0);`	22	First trip: 65 miles traveled; a quarter of the tank used. 65 / (12.0 * 1/4), rounded to the nearest integer, is returned
`car.milesPerGallon(25, 1/8.0);`	20	Second trip: 25 miles traveled; one-eighth of the tank used. (65 + 25) / (12.0 * (1/4 + 1/8))

   Write the complete class `CarMileage`. Your implementation must meet the specifications and conform to the example shown in the table.

   For additional practice, write and test a `boolean` method `canMakeTrip(int miles)`, which returns `true` if the required amount of gas for the trip (based on the average mpg) does not exceed the amount of gas left in the tank at the start of the trip plus 10%.

3. Azon is a major online retailer. Among other things in its inventory, it carries glogs. It orders glogs weekly from Glogal Enterprises, a company that manufactures glogs. Azon keeps a list of previous orders and uses a formula to predict future demand for glogs in its orders to Glogal. Glogal's owner wishes to take a two-week vacation but not lose any business. She asks Azon to find the two consecutive weeks with the smallest total number of glogs expected to be ordered. Azon will suspend orders during these two weeks and allocate their total to the left and right neighbors of the pair in the list, in equal (or, if the total is odd, almost equal) portions.

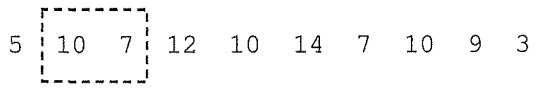
   This functionality is modeled by the class `SalesHistory`. A partial definition of `SalesHistory` is shown below.

```
public class SalesHistory
{
  /** the list of past sales */
  private ArrayList<Integer> sales;

  /**
   *  Constructs a list of sales numbers from a given array.
   *  Precondition: nums.length >= 5
   */
  public SalesHistory(int[] nums)
  { /* implementation not shown */ }

  /**
   *  Finds the two consecutive elements in sales with the
   *  smallest sum, as explained below.
   *  Preconditions: the sales list contains data;
   *                 sales.size() >= 5
   */
  public int suspendOrders()
  { /* to be implemented */ }

  /* other instance variables and methods not shown */
}
```

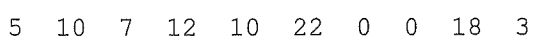
   The `suspendOrders` method searches for the two consecutive elements in `sales` with the smallest sum. The search excludes the first and the last elements. If several pairs have the same minimum value, the last one is chosen. `suspendOrders` then adds equal or almost equal halves of the sum to the neighboring elements before and after the pair. (If the portions are slightly uneven, the smaller portion is added to the preceding neighbor.) `suspendOrders` then zeroes out both elements in the pair. It returns the index of the first element in the pair.

The picture below shows an example of the `sales` list before and after the call to `suspendOrders`. The chosen pair is highlighted.

Before `suspendOrders`:



After `suspendOrders`:



As stated above, the elements at the beginning and at the end of the list are not included in the search, even though their sums of 15 and 12, respectively, are smaller than the chosen pair's sum of 17. Notice also that another pair has the sum of 17, but the later one is chosen. 8 and 9 are added to the left and right neighbors of the pair, respectively.

Complete the `suspendOrders` method below.

```
/**
 *  Finds the two consecutive elements in sales with the
 *  smallest sum. The search excludes the first and
 *  the last elements of sales. If two or more pairs
 *  of elements have the same minimum sum, takes the
 *  last such pair. Adds the found smallest sum in two
 *  equal portions (or, if odd, almost equal portions) to
 *  the neighboring element that precedes the pair and the
 *  neighboring element that follows the pair. (If the
 *  portions are slightly uneven, the smaller portion
 *  is added to the preceding neighbor.) Zeroes out
 *  both elements in the pair.
 *  Preconditions: the sales list contains data;
 *                 sales.size() >= 5
 */
public int suspendOrders()
```

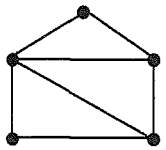
Class information for this question

```
public class SalesHistory

private ArrayList<Integer> sales)

public SalesHistory(int[] nums)
public int suspendOrders()
```

For additional practice, implement the constructor of the `SalesHistory` class.

Add a method `smooth` that replaces each element in `sales` — starting with the second one, and ending at the next-to-last one — with the average of the element and its two neighbors. Do not use any temporary arrays or lists; just save an element before overwriting it.

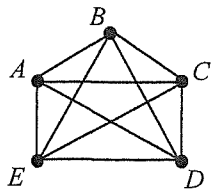
4. The picture below shows a few points connected by line segments.



Such a structure is called a *graph*; the points are called *vertices*, and the line segments are called *edges*. Graphs are useful for modeling all kinds of things: computer networks, airline routes, connections in social media apps, and board game rules and strategies, to name a few. We will consider only simple graphs in which no more than one edge connects any two vertices and a vertex cannot be connected to itself.

In some algorithms dealing with graphs, it is convenient to represent a graph as an *n*-by-*n* two-dimensional array of integers, where *n* is the number of vertices. Such a square 2D array is called the *adjacency matrix* of the graph. For the adjacency matrix named `m`, if the vertices with indices `i` and `j` are connected by an edge, then `m[i][j]` and `m[j][i]` are both set to 1; otherwise these elements of `m` are set to 0.

A simple graph is called *complete* if an edge connects every pair of its vertices. For example:

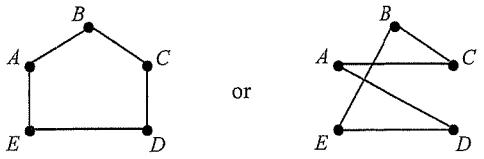


Write a `boolean` method `isComplete` that takes an adjacency matrix of a graph as a parameter and returns `true` if the corresponding graph is complete; otherwise the method returns `false`. (The adjacency matrix of a complete graph has 0's on the NW-SE diagonal and 1's everywhere else.)

Write the method `isComplete` below.

```
/**
 *  Returns true if the graph represented by the
 *  given adjacency matrix is a complete graph; otherwise
 *  returns false.
 */
public static boolean isComplete(int[][] m)
```

A graph is called a *cycle* if its vertices are arranged in one cycle with each vertex connected to exactly two neighbors. For example:



For additional practice, write a `boolean` method `isCycle` that takes the adjacency matrix of a graph and determines whether the graph is a cycle.